CONFIDENTIAL COMPUTING ENABLES USERS TO AUTHENTICATE CODE RUNNING IN TEES, BUT USERS ALSO NEED EVIDENCE THIS CODE IS TRUSTWORTHY.

Why Should I Trust Your Code?

ANTOINE DELIGNAT-LAVAUD, CÉDRIC FOURNET, KAPIL VASWANI, SYLVAN CLEBSCH, MAIK RIECHERT, MANUEL COSTA, MARK RUSSINOVICH

Now that communications and storage are encrypted by default, Confidential Computing (CC) is the next big step in making the cloud more secure: By terminating network connections to a confidential service within a hardware-isolated Trusted Execution Environment (TEE) whose memory is encrypted with keys accessible only to the processor, it is possible to protect data in use and to treat the cloud hosting infrastructure as part of the adversary, much as networking and storage infrastructures are treated today.

Consider, for example, an AI cloud service that uses a large language model to chat with users about a range of sensitive topics such as health, finances, and politics. Many users worry that these services may store their conversations and use them for malicious purposes. Can CC be leveraged to offer strong technical guarantees that the conversations will remain private?

A key piece to solving this puzzle is remote attestation: The hardware can authenticate initial TEE configurations,

including identifying the code in the TEE by its cryptographic digest and proving to a remote party that their connection terminates inside a hardware-isolated TEE (and not, say, a software emulation). This is critical, since the actual guarantees that users get depend on both hardware isolation and the code that processes their data inside the TEE. This still leaves users with a difficult decision: Should they trust this code?

A user who trusts no one could, in principle, download all source code and dependencies (including all build scripts and tools[18]) from the software providers; carefully review them to ensure there are no backdoors or vulnerabilities; then recompile the code for the service, rehash its binary, and match it against the digest attested by the TEE. This requires time and expertise, and may fail if the build is not reproducible or depends on specific versions of libraries and tools not available to all users. It is also at odds with modern software engineering, which relies on cloud platform services (such as container platforms, key-value stores, key management systems, etc.) to be scalable and competitive, rather than building everything from scratch.

On closer inspection, this is, in fact, impossible for two reasons. First, and most importantly, it expects too much of security reviews: Critical vulnerabilities in code, protocols, and even standards are still found many years after their initial deployment; understanding what code does, or just ensuring it does no harm, is fundamentally hard. To complicate matters, cloud services are updated frequently (weekly updates are common), sometimes in a hurry to deploy security patches. Hence, we must prepare

for future vulnerability disclosures by making sure the resulting patches can be disseminated and deployed quickly; it is not realistic to wait for all users to complete code reviews before deploying these patches.

Thus, for CC to become ubiquitous in the cloud, in the same way that HTTPS became the default for networking, a different, more flexible approach is needed. Although there is no guarantee that every malicious code behavior will be caught upfront, precise auditability can be guaranteed: Anyone who suspects that trust has been broken by a confidential service should be able to audit any part of its attested code base, including all updates, dependencies, policies, and tools.

To achieve this, we propose an architecture to track code provenance and to hold code providers accountable. At its core, a new Code Transparency Service (CTS) maintains a public, append-only ledger that records all code deployed for confidential services. Before registering new code, CTS automatically applies policies to enforce code-integrity properties. For example, it can enforce the use of authorized releases of library dependencies and verify that code has been compiled with specific runtime checks and analyzed by specific tools. These upfront checks prevent common supply-chain attacks.

Further policies can ensure enough evidence is recorded for audit: for example, requiring build artifacts to be escrowed outside of the control of the software provider and requiring reproducible or attested builds. By enforcing provenance and integrity, CTS provides an independent root of trust for the CC software supply chain.

As they connect to a confidential service, users can

now verify that its attested code has been successfully registered by CTS and is therefore policy-compliant and auditable. Crucially, users do not have to audit code before it is deployed, although this is supported and may be required by some users. Audits require substantial resources; thus, a limited number of organizations are expected to perform them. Audits benefit all users, however, since they all share the CTS ledger.

We argue that code transparency provides strong deterrence of malicious behaviors by making them leave indelible traces, while providing the agility needed to handle cloud-scale deployments. This approach is reminiscent of certificate transparency,[9] which helps users decide if web certificates are trustworthy. By learning the lessons of the HTTPS rollout, we hope that CC will be adopted by most services in less than a decade.

CONFIDENTIAL COMPUTING FUNDAMENTALS

For an introduction to CC, see the 2021 article, "Toward Confidential Cloud Computing," by Russinovich, et al.[15] Following are explanations of the fundamentals that together make CC possible.

Isolation and attestation

CC leverages novel CPU capabilities to create TEEs that isolate the code and data of a given task from the rest of the system, including privileged components, such as the host operating system and the hypervisor.

The TEE code can request the hardware to attest a given message (such as a public key), together with the digests of its binary image and configuration, measured when

the TEE was created. The attestation is signed with a key unique to the CPU (stored in hardware fuses) and backed by a public-key certificate for the platform (endorsed by the hardware vendor). By verifying this signature, a user can thus authenticate the TEE's code and hardware platform before trusting it with private data.

Hardware support

CPUs support CC in several form factors: Intel SGX provides subprocess TEEs (also known as *enclaves*) by extending process isolation; AMD SEV-SNP, Intel TDX, and ARM CCA provide VM-based TEEs by strengthening VM isolation. CC is also supported in high-performance accelerators, including Nvidia GPUs.

Software support

TEEs involve practical tradeoffs between usability and security, which hinge on the size and complexity of their attested Trusted Computing Base (TCB). While enclaves promote minimal software TCBs, they often require code refactoring with considerable engineering costs. VM-based isolation applies both to minimal enclave-like VMs and full-fledged legacy VMs; the latter offer better usability but lower security benefits, since legacy VMs often include a large, dynamic software TCB that depends on external agents and services.

Confidential containers

Modern cloud-native services often rely on containers and orchestration services, such as Kubernetes for their deployment, maintenance, and scalability. They

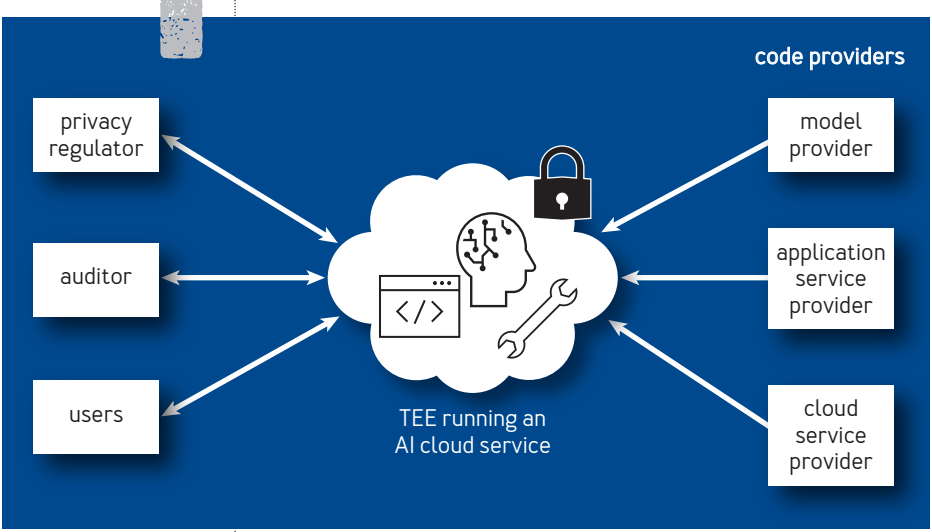
are convenient for CC, but they complicate attestation, since some trusted components—such as the guest OS kernel, the container runtime, and side containers—are managed by the Cloud Service Provider (CSP), whereas the application container is owned by the tenant. The following sample service is assumed to run in confidential containers within VM-based TEEs such as those provided in the cloud by Microsoft[11] and Google.[6]

A SAMPLE CONFIDENTIAL AI CLOUD SERVICE

Let's flesh out the sample service and present the main parties involved (see figure 1):

- A *model provider* researches, develops, and trains a machine-learning model. They want to monetize their model but worry that if they give it to others to run locally,

FIGURE 1: **SAMPLE CONFIDENTIAL AI SERVICE OPERATED IN A PUBLIC CLOUD**



it could be pirated or leak private training data. Instead, they opt to offer query access to the model in the cloud.

- An *application provider* leverages this model in their service. They want tight control of the service to provide assurances to their users that their conversations are private (e.g., not reused for training or advertising, or even exposed to third parties).
- A *privacy regulator* wants to ensure that users' conversations are processed only for the stated purpose of the service and not retained or reused by the service provider.
- An *auditor* provides independent security and privacy reviews of the service.

To address their mutual concerns, these parties agree to deploy the service in confidential containers within TEEs operated in a public cloud.

- The providers agree on the code for the service, using reputable open-source projects and documenting their dependencies. This may include code from the model provider, service provider, CSP, and third-party developers. They review and authorize the resulting container configuration. As explained later, this involves signing and registering claims about the service.
- The CSP provisions VM TEEs that load and attest this configuration.
- The model provider verifies their attestation before releasing the model decryption key to these TEEs.

CODE TRANSPARENCY SERVICE

Suppose a user wants to trust this sample AI service with a conversation about personal health. When the user's

client opens a connection to the TEE, before sending any personal data, the client should be given evidence that the code used to process the conversation is provided for the service, policy-compliant, and auditable. To achieve this, the architecture requires that all parties publish up-to-date records about the code trusted to run confidential services.

Users gain trust in these services because the evidence provides the means to hold any bad actors accountable: If any misbehavior is suspected, the permanent trace enables auditors to investigate who is responsible. This evidence may include information such as the latest-known good code and configuration; the code provenance, including versions of source projects, binary packages, and software toolchains; and their review and endorsement by independent parties.

The core new component of the architecture (in figure 2) is an attested CTS that maintains a public append-only verifiable ledger of claims signed by the other parties and that produces proofs of claim registration, referred to as *receipts*.

Issuing and registering claims

Claims are statements about a confidential service; a party issues claims by signing and registering them at the CTS. The model provider may issue a claim that records a new version of their model and accompanying software (recording, for example, their binary hashes and metadata, such as source-code tags, timestamps, and versions); the application service provider may similarly issue a claim for each version of their server; and the CSP may issue a claim

for each new version of their container runtime. More generally, these may also issue claims: CPU manufacturers for their firmware; continuous integration services for their build reports; and security experts for publishing their reviews.

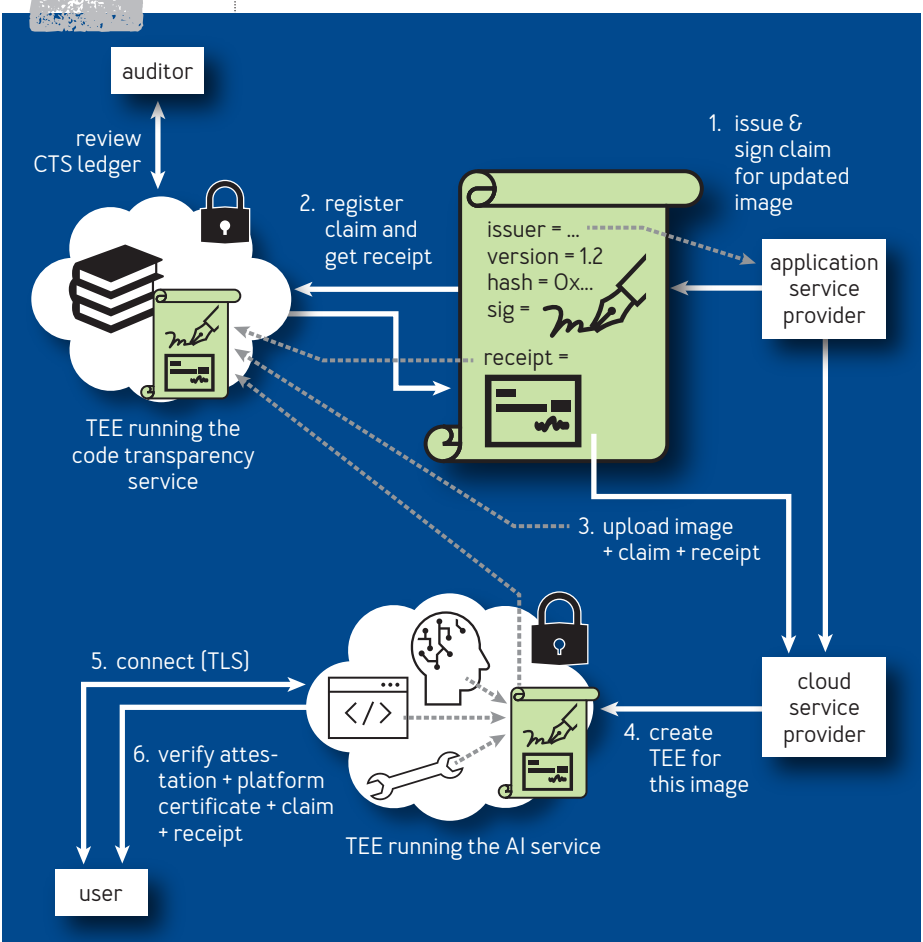
Claims may also contain policies. For example, code providers may issue configurations for reproducible builds; the model provider may issue requirements on the server configuration it supports; and the service provider may record their policies to endorse future code updates. Once registered, these policies may be applied by some clients (for example, by the model provider before releasing the model key, or by the user before sending requests) or enforced upfront by CTS before registering additional claims.

Figure 2 illustrates the workflow to deploy an updated binary image for the sample service, supported by a claim from the application service provider.

1. The service provider reviews the update, then signs a claim that endorses the hash of the updated binary image. (The review may involve additional claims that support the update, registered earlier by other actors upstream in the supply chain and verified by the service provider before signing off.)
2. It calls CTS to register this new claim; CTS applies its registration policy, then appends the claim to its transparency ledger and returns a receipt as proof of registration.
3. The service provider uploads the binary image with its claim and receipt to the CSP.
4. The CSP creates a TEE that loads and attests the new

binary image with its Transport Layer Security (TLS) key.
5. The client connects to this TEE using TLS.
6. The client receives and verifies a platform certificate,

FIGURE 2: **TRANSPARENT UPDATE FOR A SAMPLE CONFIDENTIAL AI SERVICE**



an attestation report binding the new binary hash to the server TLS key, a claim with the same hash signed by the service provider, and its receipt signed by CTS.

Before registering a claim, CTS applies a registration policy determined by the contents of its ledger. Some registration policies are generic, ensuring, for example, that the claim is well formed, its issuer is correctly identified, and its signature is valid. More advanced policies can enforce the consistency of a series of claims and even automate checks that would otherwise be performed by human auditors, by verifying auxiliary claims and attestation reports from TEEs trusted to perform these checks. Sample policies are illustrated in more detail later in the article.

Receipts: Cryptographic proofs of registration and freshness

Registration ensures that claims are globally visible and policy-compliant, and cannot be retracted or tampered with. At the end of registration, CTS produces, signs, and returns a receipt that enables anyone to verify that the claim was registered at a given time and position in the ledger. Much like the issuer signature, the receipt can be attached to the claim and distributed with it, and its verification is a local operation that does not involve communication with CTS.

Receipts are implemented using Merkle trees over the whole CTS ledger. For a given claim, a receipt consists of the tree root signed by CTS, with intermediate hashes to recompute the root from the leaf that records this claim in the tree. CTS can efficiently produce receipts by computing

the Merkle tree incrementally and signing its root only once for each batch of claims it registers.

Verification is also efficient: It entails locally recomputing the root and verifying its signature. To support systems that can verify plain signatures only, such as hardware devices authenticating their firmware at boot time, it is possible to rely on a legacy signing service that verifies the receipt before producing the signature.

Receipts also provide evidence to hold CTS itself accountable, since everyone can replay the ledger to confirm that a given receipt was correctly issued, or blame the transparency service. For example, signed receipts can be presented as proof of misbehavior in case CTS forks or corrupts the ledger.

Receipts can optionally be used to verify that a claim is up to date. This is important for preventing rollback attacks where a malicious CSP would deploy an out-of-date vulnerable version of the service. The design space to check freshness of claims is large; this article briefly describes two approaches.

One approach is to issue claims that have an expiration time, requiring the claims be renewed periodically. Another approach is to provide receipts that include proof that, at the time the receipt was produced (possibly long after the time the claim was registered), the corresponding claim had not been subsumed in the ledger by a more recent claim. This is useful to ensure that the claim is the latest in a series. For example, a receipt for our service container may prove both that the claim was registered three months ago by the service provider and was still the latest for the service one day ago, when this receipt was

produced by CTS. By daily fetching and attaching a recent receipt to the claim, the CSP thus enables users to verify that they are using the latest code for the service, with a latency of at most one day.

Authenticity and transparency

Our approach provides two integrity properties: *authenticity*, meaning every claim must have been signed by the identity key of the party that issued it; and *transparency*, meaning every claim must have been registered in the CTS append-only ledger and must have passed its registration policy at that time.

On its own, claim authenticity does not guarantee the issuer is honest. Similarly, transparency does not prevent dishonest or compromised issuers, but it holds them accountable. For example, an auditor can access the ledger and independently review the complete list of claims for all binary images endorsed by the service provider. Reputable issuers thus have incentive to carefully review their statements before signing and registering them. Similarly, a reputable CTS has incentive to securely manage its ledger, as any inconsistency can be pinpointed by the auditor.

By maintaining a complete record of registered claims, CTS also defends against advanced targeted attacks that may involve bad actors issuing specially crafted claims to deceive some users. For example, imagine the service provider or the CSP (possibly coerced by a local authority) attempts to eavesdrop on a user by directing the user's requests to a TEE running a malicious variant of the service that silently forwards user input to a third party.

This malicious code may even be presented as a legitimate security update, both attested by the TEE and supported by a claim signed by the provider. This privileged attack is hard for the user to detect, since the service requests and responses are unchanged.

If the user requires a registration receipt and verifies it, however, the malicious claim must have been publicly registered and so may be spotted at any time by an auditor reviewing the service update history. The auditor then has signed evidence to blame the service provider. This strikes a good balance between serviceability and security: Bad actors are deterred by the risk that they will eventually be caught based on registered evidence, and, at the same time, this supports cloud agility and scale because users and auditors do not need to audit code updates before deployment.

This notion of code transparency is inspired by certificate transparency[9] and many related efforts to maintain an independent, consistent record of signed statements produced by semi-trusted authorities and consumed by multiple parties that may not otherwise interact with one another. Certificate transparency has proved effective at compelling CAs (certificate authorities) to follow CA/Browser Forum guidelines or risk being removed from root programs (which has indeed occurred) and has scaled to more than 10 billion logged certificates. Transparency logs have also been usefully applied to, for example, public-key records,[10] signature delegation,[12] supply-chain policies,[8] software packages,[19] firmware updates,[1] distributed builds,[13] and multiparty computations.[7]

THREAT ANALYSIS

As in an Agatha Christie novel, any of the parties involved in this sample confidential service might be malicious. Let's analyze the resulting risks of service compromise.

CC allows the focus to be on the TEEs that run the service, while the rest of the hosting infrastructure is treated as untrusted. In particular, the CSP is trusted only for the code it contributes to the TEEs (such as utility VMs for the service containers), recorded in claims issued by the CSP and subject to code attestation and transparency. The CSP is otherwise untrusted; it controls the creation of TEEs and their untrusted network and storage; hence, it may easily degrade availability and quality of service. It may also, for example, delay or block the deployment of a critical service update. But it cannot break the integrity or confidentiality of the TEE code and data, and thus (since all client communications are protected by TLS) of their requests and responses.

We therefore focus on the potential causes and consequences of TEE compromise. Although TEEs significantly increase platform security, they are still vulnerable to attacks beyond their threat model (such as advanced physical attacks) and to defects in their design and implementations. While a platform-specific discussion is out of scope, we distinguish two cases:

- A corrupt TEE hardware platform (e.g., a specific CPU or even a whole CPU family) may issue attestation reports with arbitrary contents. This may be caused by physical attacks, critical firmware exploits, or rogue platform certificates. The most severe attacks may enable rogue attestation reports with arbitrary code identity.

These attacks may be mitigated by enforcing restrictive attestation-verification policies (e.g., by whitelisting CPUs located in each cloud datacenter and blacklisting defective firmware versions). Transparency logs may also help detect and deter malicious platform providers by holding them accountable for their firmware and platform certificates and by providing incentive for them to patch defects quickly.

➡ A corrupt TEE instance may misuse attestation reports—for example, by signing arbitrary statements with its attested key, without being able to modify its attested code identity and initial configuration. Because of the diversity and complexity of software running inside TEEs, this case is expected to be the most common. Common causes include Trojan code running within the TEE, microarchitectural side-channel attacks leading to key compromise, or just buggy code enabling buffer overruns and privilege escalation.

We further focus on this second case where, crucially, the TEE compromise can be traced back to its attested code. It is also assumed that any relying party will verify a transparency receipt for this code before trusting the TEE. This ensures that CTS must have registered claims that (erroneously) endorse this code, and, hence, that the compromise can be traced back to the claim issuer.

The main goal here is to provide auditability in *all compromise scenarios*: It should be possible to determine which signing party is at fault based on the claims recorded in the CTS ledger or—in case the ledger is unavailable or has been tampered with—on the claims and receipts kept by other parties.

Let's begin with bad claim issuers (assuming CTS is trusted), then discuss potential attacks involving CTS.

Bad claim issuers

A bad issuer may sign a claim with arbitrary payload. This may be caused by honest mistakes (such as programming errors), malicious intent (such as inserting a backdoor), or attacks against the issuer (such as compromised signing keys). In all cases, once the claim has been registered, auditing can blame the issuer on the basis of a bad payload. In many cases, other claims may provide a finer picture, (e.g., identifying both a developer who committed dangerous code and a software manager who included it in a release).

CTS attacks

As a confidential service, CTS is subject to all potential TEE attacks (and mitigations) described above. While we expect the CTS code base and governance to be carefully reviewed and audited, the impact of their compromise still needs to be considered.

A corrupt TEE running CTS may register claims that do not meet their registration policy (e.g., claims with issuer identities and signatures that do not verify) and may issue receipts that do not match its ledger (e.g., for unregistered or out-of-date claims). To mitigate these attacks, the CTS code base and its governance are recorded in a series of claims in its ledger. Given access to the ledger, an auditor can review them in detail; it can also replay the registrations recorded in the ledger to detect any error or inconsistency; similarly, it can check that a collection of

claims and receipts is consistent with the ledger and blame CTS otherwise.

The CSP may also degrade CTS availability. In particular, it may prevent access to the ledger and thus limit the scope of audits. The CSP will be blamed for such outages, which may be mitigated by replicating and archiving the ledger in different trust domains. In general, auditing depends on the quality of the records in the claims.

Code-transparency policies can help ensure that the records are sufficiently detailed. Precise auditing may also depend on additional information held by other services. In the example, we use CTS to keep track of source-code releases by recording their tags and cryptographic digests but still rely on the availability of git repositories to track software vulnerabilities to individual commits.

TRANSPARENCY POLICIES

Recall that CTS enforces predefined registration policies before producing receipts; this prevents some mistakes and attacks upfront. CTS also enables any auditing policies to be applied later, to detect and deter (but not prevent) more sophisticated or less predictable attacks. Following is a discussion of several code-transparency policies and how they can be enforced by CTS, auditors, and auxiliary TEEs.

As a base registration policy, CTS always ensures the claim is well formed and includes the issuer's long-term identifier, key information, and signature; it verifies the key is valid for the issuer at that time, and the signature is correct for the claim. This is important to protect users who will verify only the receipt.

CTS may apply additional registration policies, as

illustrated on two use cases:

- To register a claim that records a source release of a given open-source project—recording, for example, a project URL, a git commit hash, and a release tag—the policy may verify the release is on the main stable branch of the project, the tag is consistent with those in prior claims (e.g., enforcing semantic versioning or increasing security version numbers), the code repository is correctly configured (e.g., requiring two-factor authentication), every intermediate commit is well formed and signed, and the release itself is signed by an authorized project manager. The policy may additionally verify auxiliary claims (e.g., source-code claims for the project submodules or dependencies) and claims produced by code-scanning or code-verification services.
- To register a claim that records a successful build step—recording, for example, its source-code dependencies, build environment, and the resulting binary packages or binary images—the policy may verify that the claim is issued by a trusted build service and may additionally verify a claim for each of these inputs. While not strictly necessary, requiring the build tools specified in the claim to be deterministic significantly reduces the need to trust the build service (since the auditor can rebuild and blame the build service if it yields different binaries). The policy may require that the build run in a designated build container within a TEE that issues a build claim and a supporting attestation report, significantly increasing trust and auditability of the build service.

These sample policies involve claims that record locations and digests for materials stored outside the

ledger, such as source trees, containers, and binaries. Inasmuch as precise auditing depends on these materials, a registration policy may additionally check that these materials are stored in reputable storage (such as a public source-code repository or container registry) with adequate replication and read access for auditors (using, for example, different clouds or trust domains). This is particularly important to ensure transparency even if some of the code is not publicly available.

Many policies are too complex to be directly enforced by CTS without bloating its own trusted code base: CTS should be able to validate declarative policies and their cryptographic materials (including signatures, claims, receipts, and attestation reports) but not run large programmable build tasks. For such advanced use cases, CTS can instead use registration policies that leverage CC to delegate the execution of complex tasks to auxiliary TEEs. We implemented this approach to automatically build and register the attested code for the sample AI service and for bootstrapping CTS itself.

The main idea is to let issuers delegate tasks they would normally perform on their own before issuing their claims (e.g., for releasing a new source version of their software) or for building a binary image using newly released source code.

To this end, these parties issue and sign instead a delegation-policy claim that specifies the delegated task, the TEE configuration that they trust to run the task on their behalf (e.g., the containerized build environment to use), the attestation-verification policy to use, and the template of the resulting claim to be issued by the TEE once the task completes.

When the CSP needs to run a task—for example, to update the service after the release of a security patch—it creates a TEE based on the configuration specified in the registered delegation policy for this task. This TEE creates an ephemeral signing key; generates an attestation report that binds the corresponding public key to its code identity; runs the task; and (assuming the task completes successfully) issues a claim for the updated binary image it has just compiled, together with its own attestation report and build log; signs the claim with its ephemeral attested key; and finally registers the claim at the CTS.

When presented with a claim issued by a TEE, CTS verifies its attestation report and attested configuration against the latest registered delegation policy for the task and, if all verification succeeds, registers the delegated claim.

Once delegation policy claims have been written and registered by "human" parties, the whole process can be automated by untrusted parties such as the CSP, and yet a claim for the resulting binary image will be registered only if it passes the previously registered policies. Hence, users who verify the resulting delegated claim are guaranteed before using their service that its code complies with all policies registered at the transparency service.

IMPLEMENTING CTS

Finally, let's outline the ongoing standardization efforts and our implementation of CTS, based on draft RFCs for its claim formats and protocols, and on the Confidential Consortium Framework (CCF[4]) to provide an append-only, verifiable, tamper-evident ledger of claims enforced by

SGX enclaves. This enables us to bootstrap trust in CTS using attestation, transparency, and auditing for its own code base.

Standardized claims formats

To be adopted by software providers, code transparency needs a broad agreement on common formats and protocols for issuing, registering, and verifying claims. Specifying them and ensuring their interoperability is the charter of the IETF SCITT (Supply Chain Integrity, Transparency, and Trust) Working Group[2].

SCITT provides generic support for exchanging transparent claims along supply chains; it specifies a transparency service that records claims but does not interpret their payloads (which are important but usually specific to each supply chain). Independent standardization efforts aim to provide standard formats for common claim payloads, such as SBOM (software bills of materials).[5,17]

SCITT represents claims as CBOR (Concise Binary Object Representation)-encoded signed envelopes with specific headers that must be understood by all parties. Hence, standardized headers record the long-term identity of the claim issuer, represented as a W3C DID (decentralized identifier)[3] (e.g., the service provider) and the purpose of the claim (e.g., authorize a binary image for a given service). While a SCITT transparency service is mostly concerned about these headers, issuers that sign claims and verifiers that make trust decisions based on claims should also parse and understand their payloads. For example, our application of SCITT to CC involves different types of claims for source-code releases, build

policies, container images, container configurations, and SGX binary images.

An attested CTS implementation

Our implementation combines TEEs (with a transparent, attested, and relatively small TCB) with decentralized ledger technology from CCF.[4] A technical report describes its design and evaluation.[20]

CCF runs a consensus protocol between multiple TEEs to ensure the ledger is persisted and the service can withstand individual TEE failures; it also limits trust in the CSP by allowing a consortium of members to vote on important governance decisions, such as changes to the security policies it enforces.

Our CTS prototype[16] is a CCF application deployed in SGX enclaves within Azure DCsv3 series VMs. The application code consists of about 3,500 lines of C++ code, plus about 3,000 lines of Python code for client tooling, notably a command-line tool to format, sign, and register claims. The service exposes REST (representational state transfer) endpoints for registration and receipts. It supports flexible registration policies programmed in Typescript and registered in specific claims. It can register 1,550 claims per second and can produce 5,100 receipts per second and per thread.

CTS relies on a combination of transparent claims and CCF governance for securing its own code updates. Once authorized by a claim issued by the CTS operator, registered by CTS, and verified by its governance code-update policy (written in Typescript and also recorded in the ledger), the TEE nodes that jointly implement the

service can be gradually replaced by new TEE nodes running its updated code, ready to provide up-to-date attestation reports and transparency claims to CTS clients. As outlined in the earlier discussion of policies, the process can be fully automated for building and deploying updates that do not involve a change of policy.

We evaluated our approach by using our CTS prototype to automatically record a transparent update and attested build history for a series of existing open-source projects, including OpenSSL, Triton,[21] Open Enclave,[14] CCF,[4] and CTS itself.[16] Their builds are complex and involve large TEEs (32-core SEV-SNP VMs with 64GB of RAM) downloading hundreds of packages before finally signing and registering claims for the resulting binary images. On the other hand, they required modifying only a few lines of Dockerfile to run them within an attested container.

SUMMARY

CC makes it possible for users to authenticate code running in TEEs but not to determine if it is trustworthy. This is a hard problem because the TCB of a confidential service may be large, complex, frequently updated, and exposed to attacks along its software supply chain.

We have shown how to keep track of any code and policy that contributes to the TCB of a confidential service with precisely defined, limited, and transparent trust assumptions in its software providers. This does not quite solve the problem, but does make it tractable, enabling all parties on the CC supply chain to gain more trust in the code they depend on and, in case of attack, to hold bad actors accountable based on transparent signed evidence.

Our code transparency service can automate the process with strong CC safeguards of its own, but its success in practice will depend on its broad adoption by the technical community of software developers, service providers, cloud operators, and security experts. This is part of a larger standardization effort at the IETF and elsewhere.

From this long-term viewpoint, it is interesting to look back at the fumbling days of SSL (Secure Sockets Layer), where actors expressed many similar concerns to those discussed today: Will HTTPS/TEEs make my website slower, or harder, to operate? What if my signing keys are compromised? Who sets the certification/attestation policies and audits them? Who is responsible? How to identify bad actors?

Ultimately, a combination of new open standards and protocols with pressure from user privacy advocacy groups gradually moved the needle for communications security. CC will likely follow the same path.

References

1. Al-Bassam, M., Meiklejohn, S. 2018. Contour: a practical system for binary transparency. In *Data Privacy Management, Cryptocurrencies and Blockchain Technology*, 94–110. Springer; https://link.springer.com/chapter/10.1007/978-3-030-00305-0_8.
2. Birkholz, H., Delignat-Lavaud, A., Fournet, C., Deshpande, Y., Lasker, S. 2022. An architecture for trustworthy and transparent digital supply chains. IETF SCITT Working Group; https://datatracker.ietf.org/doc/draft-ietf-scitt-architecture/.

3. Brunner, C., Gallersdörfer, U., Knirsch, F., Engel, D., Matthes, F. 2020. DID and VC: untangling decentralized identifiers and verifiable credentials for the web of trust. In *Proceedings of the Third International Conference on Blockchain Technology and Applications*, 61–66; https://dl.acm.org/doi/abs/10.1145/3446983.3446992.
4. Confidential Consortium Framework. Microsoft. GitHub; https://github.com/microsoft/CCF.
5. CycloneDX SBOM standard. 2023. CycloneDX; https://cyclonedx.org.
6. Damlaj, I., Saboori, A. 2020. A deeper dive into confidential GKE nodes. Google; https://cloud.google.com/blog/products/identity-security/confidential-gke-nodes-now-available.
7. Dauterman, E., Fang, V., Crooks, N., Popa, R. A. 2022. Reflections on trusting distributed trust. In *Proceedings of the 21st ACM Workshop on Hot Topics in Networks* (HotNets), 38–45; https://dl.acm.org/doi/10.1145/3563766.3564089.
8. Ferraiuolo, A., Behjati, R., Santoro, T. Laurie, B. 2022. Policy transparency: authorization logic meets general transparency to prove software supply chain integrity. In *Proceedings of the ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses*, 3–13; https://dl.acm.org/doi/10.1145/3560835.3564549.
9. Laurie, B. 2014. Certificate transparency. *Communications of the ACM* 57(10), 40–46; https://dl.acm.org/doi/abs/10.1145/2659897.
10 Melara, M. S., Blankstein, A., Bonneau, J., Felten,

E. W., Freedman, M. J. 2015. CONIKS: bringing key transparency to end users. In *Proceedings of the 24th Usenix Security Symposium*; https://www.usenix.org/system/files/conference/usenixsecurity15/sec15-paper-melara.pdf.

11. Microsoft. 2023. Confidential containers on Azure container instances (ACI); https://learn.microsoft.com/en-us/azure/container-instances/container-instances-confidential-overview.
12. Newman, Z., Meyers, J. S., Torres-Arias, S. 2022. Sigstore: software signing for everybody. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*, 2353–2367; https://dl.acm.org/doi/10.1145/3548606.3560596.
13. Nikitin, K., Kokoris-Kogias, E., Jovanovic, P. Gailly, N., Gasser, L., Khoffi, I., Cappos, J., Ford, B. 2017. CHAINIAC: proactive software-update transparency via collectively signed skipchains and verified build. 26th Usenix Security Symposium; https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/nikitin.
14. Open Enclave SDK. GitHub; https://github.com/openenclave/openenclave.
15. Russinovich, M., Costa, M. Fournet, C. Chisnall, D., Delignat-Lavaud, A., Clebsch, S., Vaswani, K., Bhatia, V. 2021. Toward confidential cloud computing. *Communications of the ACM* 64(8), 54–61; https://cacm.acm.org/magazines/2021/6/252824-toward-confidential-cloud-computing/abstract.
16. SCITT service prototype based on CCF. 2023. Microsoft. GitHub; https://github.com/microsoft/scitt-ccf-ledger.

17. Stewart, K., Odence, P. Rockett, E. 2010. Software package data exchange (SPDX) specification. *International Free and Open Source Software Law Review* 2(2), 191–196; https://www.jolts.world/index.php/jolts/article/view/45.
18. Thompson, K. 1984. Reflections on trusting trust. *Communications of the ACM* 27(8), 761–763; https://dl.acm.org/doi/10.1145/358198.358210.
19. Torres-Arias, S., Afzali, H., Kuppusamy, T. K., Curtmola, R., Cappos, J. 2019. in-toto: providing farm-to-table guarantees for bits and bytes. In *Proceedings of the 28th Usenix Security Symposium*; https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias.
20. Transparent code updates for confidential computing. Draft technical report. https://www.microsoft.com/research/group/azure-research/.
21. Triton inference server. GitHub; https://github.com/triton-inference-server.

Antoine Delignat-Lavaud *is a principal researcher in Azure Research at Microsoft and is researching protocols and systems for confidential cloud services built on hardware-security guarantees.*

Cédric Fournet *is a senior principal research manager in Azure Research at Microsoft. He is interested in security, privacy, cryptography, distributed programming, and formally verified systems.*

Kapil Vaswani *is a senior principal researcher in Azure Research at Microsoft. His research focuses on building*

secure, robust, and transparent systems. He has pioneered work on secure databases using trusted hardware and new forms of trusted hardware.

Sylvan Clebsch *is a senior principal research engineer in Azure Research at Microsoft. His work includes the Confidential Consortium Framework, Verona, and the Pony programming language.*

Maik Riechert *is a senior research engineer in Microsoft Research. He is interested in supply-chain security, privacy, and machine learning.*

Manuel Costa *is a partner research manager at Microsoft, where he leads Azure Research. He is interested in advancing the state of the art in security and privacy, operating systems, and programming languages.*

Mark Russinovich *is CTO of Microsoft Azure, where he leads technical strategy and architecture for Microsoft's cloud computing platform.*

CONTENTS